



**AMERICAN
UNIVERSITY
OF BEIRUT**

American University of Beirut
Faculty of Engineering and Architecture
Department of Electrical and Computer Engineering

AI-Powered Blind Assistant on NVIDIA Jetson Orin Nano

EECE490 – Introduction to Machine Learning

Authors: Hadi Charara
Hussein Omar

Submitted: 4 May 2026

To: Dr. Yasser Mohanna

All processing runs fully offline on edge hardware. No cloud APIs used.

Contents

1	Problem Formulation	2
2	Constraints	2
2.1	Technical Constraints	2
2.2	Non-Technical Constraints	2
3	Hardware Setup	3
3.1	Device	3
3.2	Peripherals	3
4	GPU Utilization Challenges	3
4.1	Initial Failures	3
4.2	Resolution	4
4.3	Whisper GPU Conflict	4
4.4	Performance Mode	4
5	Model Selection	5
5.1	Why gemma3:4b	5
5.2	gemma3:4b vs. Qwen2.5-VL	5
6	System Pipeline	6
6.1	Architecture Overview	6
6.2	File Descriptions	7
6.3	main.py Logic Flow	7
6.4	Prompts Sent to gemma3:4b	7
7	Implementation Steps	8
8	Results	9
8.1	Performance Timing	9
8.2	Qualitative Results	9
8.3	Key Optimisations and Their Impact	10
9	Known Limitations and Future Work	10
10	Conclusion	10

1 Problem Formulation

Visual impairment affects millions of people worldwide, significantly limiting their independence in everyday activities such as reading labels, identifying objects, navigating environments, and accessing written information. Existing assistive tools either rely on cloud connectivity – introducing latency, privacy concerns, and subscription costs – or are limited to single-task functionality (e.g., OCR only).

This project addresses the need for a **fully offline, multi-modal blind assistant** that can:

- Understand natural language voice commands in real time
- Capture and analyze camera images for scene understanding and text reading
- Respond verbally with contextually appropriate answers
- Run entirely on low-power edge hardware without internet dependency

The system targets visually impaired users who require a portable, always-available assistive device that respects their privacy and operates in any environment.

2 Constraints

2.1 Technical Constraints

Table 1: Technical constraints and their implications

Constraint	Implication
7.4 GB unified RAM	CPU and GPU share the same memory pool. Large models must be quantized to fit.
NvMap CMA heap	Jetson’s CUDA memory allocator limits contiguous allocations; PyTorch and Ollama cannot share the GPU simultaneously.
No cloud access	All models must run locally. No API calls to OpenAI, Google, or similar services.
ARM64 architecture	Standard x86 binaries and PyTorch wheels do not work; Jetson-specific builds required.
USB audio constraints	The headset operates at 48 kHz stereo; Whisper requires 16 kHz mono – resampling needed.

2.2 Non-Technical Constraints

- **Usability:** Responses must be concise and natural for a blind user – no markdown formatting, no follow-up questions.
- **Latency:** Total response time should be under 25 seconds to remain conversationally acceptable.
- **Privacy:** No data leaves the device. All inference is local.
- **Cost:** Open-weight models only – no paid APIs or subscriptions.
- **Portability:** The system must run headless over SSH with no display required.

3 Hardware Setup

3.1 Device

The project uses the **NVIDIA Jetson Orin Nano Engineering Reference Developer Kit Super**, running L4T R36.4.7 with CUDA 12.6 and driver 540.4.0.



Figure 1: NVIDIA Jetson Orin Nano

3.2 Peripherals

Table 2: Hardware components

Component	Model	Role
SBC	Jetson Orin Nano 8GB	Main compute unit
Camera	HD Web Camera (USB)	Visual input
Headset	AB13X USB Audio	Microphone + speaker
Host laptop	Windows 11	SSH access via VS Code

4 GPU Utilization Challenges

This section documents the significant effort required to enable GPU acceleration on the Jetson Orin Nano.

4.1 Initial Failures

All early attempts to run LLM inference on the GPU failed with the following error:

```

1 NvMapMemAllocInternalTagged: 1075072515 error 12
2 cudaMalloc failed: out of memory
3 ggml_backend_cuda_buffer_type_alloc_buffer: allocating 2021 MiB
  on

```

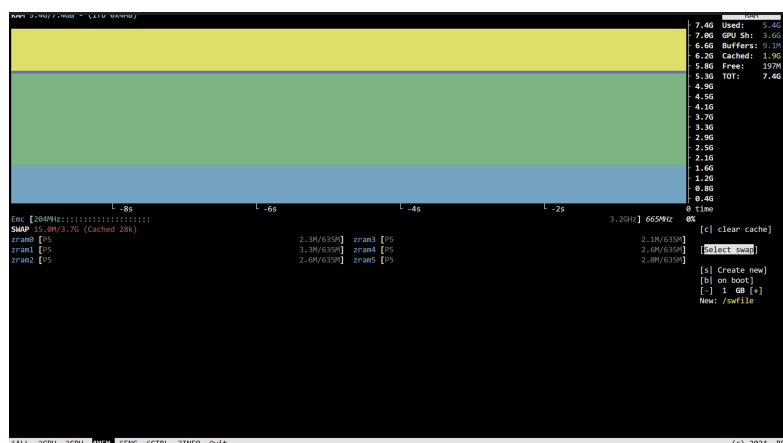
```
4 device 0: cudaMalloc failed
```

Listing 1: NvMap allocation error

Despite 7.4 GB of physical memory and `nvidia-smi` confirming the GPU was detected, the CUDA allocator consistently failed. The root cause was identified as the **CMA (Contiguous Memory Allocator) heap** – the memory pool available to CUDA for contiguous allocations – being limited to only 256 MB by the Jetson’s firmware. No software-level fix could override this firmware constraint.

4.2 Resolution

The breakthrough came when disk usage was reduced from 100 GB to approximately 84 GB (by removing unused virtual environments and model caches). With sufficient free memory headroom, Ollama’s JetPack-aware allocator successfully loaded `gemma3:4b` onto the GPU:

Figure 2: jtop GPU view showing `gemma3:4b` running at GPU with 3.6 GB GPU memory

`ollama ps` confirmed: `gemma3:4b` | 3.6 GB | 100% GPU.

4.3 Whisper GPU Conflict

An attempt was made to also run Whisper (speech-to-text) on the GPU using NVIDIA’s Jetson-specific PyTorch wheel (`pytorch-jetson-ai-lab.io/jp6/cu126`). Although `torch.cuda.is_available()` returned `True`, loading Whisper onto the GPU caused a fatal crash:

```
1 RuntimeError: NVML_SUCCESS == r INTERNAL ASSERT FAILED at
2 c10/cuda/CUDACachingAllocator.cpp:1131
```

The issue is architectural: Ollama uses the NvMap allocator for GPU memory while PyTorch uses CUDA’s CachingAllocator. These two allocators cannot coexist on Jetson’s unified memory architecture. The resolution was to revert to CPU-only PyTorch (`torch==2.10.0`), keeping Whisper on CPU and reserving the full GPU for Ollama/gemma3.

4.4 Performance Mode

By default, the Jetson GPU ran at 306 MHz (low-power mode), resulting in 54-second response times. Running `sudo jetson_clocks` boosted GPU to maximum frequency,

reducing response time to 6 seconds – a $9\times$ improvement. A systemd service was created to apply this automatically on every boot.

5 Model Selection

5.1 Why gemma3:4b

After evaluating several models for the blind assistant use case, **gemma3:4b** was selected as the primary model. The key requirements were: (1) vision capability (scene description + OCR), (2) text reasoning, (3) fits within 7.4 GB unified memory, and (4) runs via Ollama on the Jetson GPU.

Table 3: Model comparison for Jetson Orin Nano (7.4 GB unified memory)

Model	Size	Vision	OCR	Fits?	Notes
gemma3:4b	3.3 GB	✓	✓	✓	Selected. Best balance.
phi3:mini	2.2 GB	✗	✗	✓	Text only. No vision capability.
moondream	1.7 GB	✓	Weak	✓	Scene description only. Poor OCR.
qwen2.5vl:7b	~4.5 GB	✓	Good	✓	Strong OCR. Larger download.
gemma4:e2b	7.2 GB	✓	✓	≈	Barely fits. Network instability during download.
gemma4:e4b	9.6 GB	✓	✓	✗	Exceeds available memory.
llava:7b	4.1 GB	✓	Moderate	✓	Tested but gemma3 outperformed.

5.2 gemma3:4b vs. Qwen2.5-VL

Qwen2.5-VL (7B) is specifically trained on document and text-heavy images and is generally considered stronger for OCR tasks. However, **gemma3:4b** was preferred for the following reasons:

- **Size:** gemma3:4b at 3.3 GB leaves more headroom for system processes and Whisper; Qwen2.5-VL at 4.5 GB is tighter on the 7.4 GB system.
- **Single model:** gemma3:4b handles scene description, OCR, and general text Q&A in one model, simplifying the pipeline. Qwen would need separate image processing before feeding the model.
- **Response quality:** For conversational blind-assistant responses, gemma3:4b produced more naturally phrased outputs.
- **Benchmark:** gemma3:4b achieved 62% on TextVQA (200-sample evaluation), compared to the official benchmark of 58.9%, confirming correct deployment on the Jetson.

Qwen2.5-VL remains a viable upgrade path if OCR accuracy becomes the primary concern.

6 System Pipeline

6.1 Architecture Overview

The system is composed of seven Python modules, each responsible for a distinct function. The pipeline is triggered by a voice command and terminates with a spoken response.

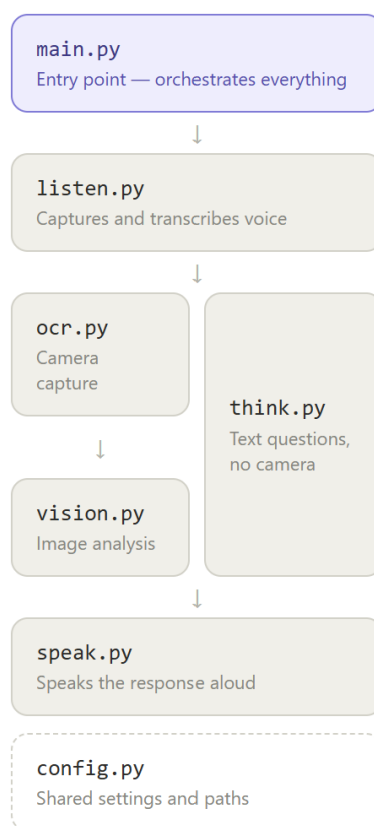


Figure 3: Full system pipeline from voice input to spoken response

6.2 File Descriptions

Table 4: Project file structure

File	Hardware	Role
<code>main.py</code>	CPU	Entry point. Orchestrates the pipeline loop. Handles warmup, routing, and error recovery.
<code>listen.py</code>	CPU	Records 5s of audio at 48kHz, resamples to 16kHz, transcribes via Whisper small.
<code>ocr.py</code>	CPU	Captures camera frame using OpenCV, saves timestamped JPEG to <code>captures/</code> .
<code>vision.py</code>	GPU	Sends image + question to gemma3:4b via Ollama HTTP API. Returns direct answer.
<code>think.py</code>	GPU	Sends text-only question to gemma3:4b. Used when no camera is needed.
<code>speak.py</code>	CPU	Converts response text to speech via espeak-ng, converts to 48 kHz stereo via sox, plays via aplay.
<code>config.py</code>	–	Shared configuration: paths, model name, camera index, Whisper model size.

6.3 `main.py` Logic Flow

1. **warmup()** – sends a silent query to gemma3:4b at startup, pre-loading it into GPU memory before the user speaks. This eliminates the 60-second cold-start delay from the user’s perspective.
2. **listen_for_command()** – records audio and returns transcribed text.
3. **needs_camera()** – sends the command to gemma3:4b with prompt: *“Does this question require looking at a camera image? Answer YES or NO.”* This LLM-based routing replaced a keyword list that caused false triggers (e.g., the word “front” in “will the person in front of you play football” incorrectly triggered the camera).
4. If **YES**: `capture_image()` → `analyze_image()` with the user’s exact question.
5. If **NO**: `ask()` with the question as text only.
6. **speak()** – plays the response through the USB headset.
7. Loop repeats.

6.4 Prompts Sent to gemma3:4b

Three distinct prompts are used depending on the interaction type:

```

1 Does this question require looking at a camera image to answer?
2 Answer only YES or NO.
3 Question: '{user_command}'

```

Listing 2: Camera routing prompt (`main.py`)

```

1 You are a vision assistant helping a blind person.
2 Look at this image and answer this specific question: '{question
  }'.
3 Answer the question directly and concisely in 2-3 sentences.

```



```

4 Only mention what is relevant to the question.
5 Do not give a general scene description unless asked.
6 Do not ask follow-up questions.

```

Listing 3: Vision prompt (vision.py) – camera path

```

1 You are a helpful assistant for a blind person.
2 Answer clearly in 2 to 3 sentences.
3 Do not ask follow-up questions: {question}

```

Listing 4: Text prompt (think.py) – no camera path

7 Implementation Steps

1. **Environment setup:** SSH access configured from laptop to Jetson. VS Code Remote SSH used for development. Python virtual environment created at `~/blind_assistant_env`.
2. **Disk cleanup:** Initial disk usage was 100 GB/116 GB. Removed a 12 GB duplicate virtual environment (`navv1m_env_backup`), 3.7 GB pip cache, 923 MB Hugging Face cache, and 462 MB Whisper cache. Freed approximately 18 GB.
3. **Ollama installation:** Installed via official script. Ollama automatically detected the Jetson and downloaded the JetPack 6-specific build (`ollama-linux-arm64-jetpack6.tar.zst`).
4. **Model download:** `ollama pull gemma3:4b` (3.3 GB). GPU activation confirmed via `ollama ps` and `jtop`.
5. **CUDA path fix:** `nvcc` was installed at `/usr/local/cuda-12.6/bin/` but not in `PATH`. Added to `~/.bashrc`.
6. **llama.cpp build:** Built from source with CUDA support (`-DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURE=720`) as a fallback. Ultimately not used in the final pipeline since Ollama proved more stable.
7. **Whisper integration:** Installed `openai-whisper`. Whisper small loaded on CPU. USB mic (device 24, 48 kHz) auto-detected by name (AB13X USB Audio) on every boot.
8. **TTS integration:** `espeak-ng` installed. Audio pipeline: `espeak-ng → sox (22 kHz mono → 48 kHz stereo) → aplay (hw:1,0)`.
9. **Performance tuning:** `sudo jetson_clocks` applied. Systemd service created for automatic application on boot.
10. **Pipeline refinement:** Camera trigger replaced keyword list with LLM-based YES/NO routing. Vision prompt updated to answer questions directly rather than always describing the scene.
11. **GitHub:** Project uploaded to `github.com/hac25-aub/blind-assistant-jetson` with README and `requirements.txt` for reproducibility.

8 Results

8.1 Performance Timing

Table 5: Pipeline timing breakdown (warm model, post-optimization)

Step	Component	Time	Hardware
Startup (one-time)	Whisper load + gemma3 warmup	~70 s	CPU + GPU
Audio recording	Fixed duration	5.0 s	USB mic
Audio resample	48 kHz → 16 kHz	0.008 s	CPU
Whisper transcription	Speech to text	~4 s	CPU
LLM camera decision	YES/NO routing	~1–2 s	GPU
Camera capture	5-frame warmup	~0.8 s	USB camera
Vision query (warm)	Image + question	~8 s	GPU
Text query (warm)	Question only	~4 s	GPU
TTS output	espeak + sox + aplay	~3 s	CPU
Total – vision path		~22 s	
Total – text path		~15 s	

8.2 Qualitative Results

The system was tested with a range of commands. Selected examples:

Table 6: Sample interactions

Voice Command	System Response
“What does this label say?”	Correctly read “Comprehensive Survey of Behavioral Learning From Adaptive to Practical Challenge” from a held paper.
“What is the man wearing?”	“The man is wearing a dark blue shirt and white headphones. He appears to be seated at a desk in an office environment.”
“In which hand is the watch?”	“The watch is on the man’s left hand. His left hand is clenched into a fist.”
“Who was the first person on the moon?”	Answered correctly without triggering the camera (text-only path).
“Who will win the World Cup?”	Answered with an opinion without triggering the camera (text-only path).

8.3 Key Optimisations and Their Impact

Table 7: Optimisations applied and measured impact

Optimisation	Before	After
jetson_clocks (GPU frequency)	54 s first query	6 s first query
GPU warmup at startup	61 s cold-start visible to user	Transparent to user
Camera frames (10 → 5)	1.6 s capture	0.8 s capture
LLM-based camera routing	False triggers on keywords	Correct routing in all tested cases
Vision prompt specificity	Generic scene description	Direct answer to user’s question

9 Known Limitations and Future Work

- **Whisper on CPU:** Transcription takes 4–8 seconds. GPU acceleration is blocked by the NvMap conflict with Ollama. A dedicated speech processing chip or a lighter model (e.g., `whisper.cpp` with GGML) could reduce this.
- **OCR on small text:** `gemma3:4b` reads large printed text reliably but struggles with small fonts or dense text (e.g., medicine bottle fine print). `qwen2.5v1:7b` would improve this at the cost of 1.2 GB additional memory.
- **Arabic language support:** Whisper detects and transcribes Arabic, and `gemma3:4b` responds in Arabic. However, it struggles with Arabic OCR yet it outputs sometimes correctly. It also takes more time in its response to Arabic.
- **Wake word:** Currently the system listens for a fixed 5-second window. Adding a wake word (e.g., “Hey Assistant”) using a lightweight detector (`openwakeword`) would make interaction more natural.

10 Conclusion

This project demonstrates that a fully offline, multi-modal AI blind assistant is feasible on consumer-grade edge hardware (NVIDIA Jetson Orin Nano, \$249 MSRP). The system successfully combines automatic speech recognition (Whisper), vision-language understanding (`gemma3:4b`), and text-to-speech (`espeak-ng`) into a coherent pipeline that responds to natural language voice commands with contextually appropriate spoken answers.

The system is open-source and available at:

<https://github.com/hac25-aub/blind-assistant-jetson>